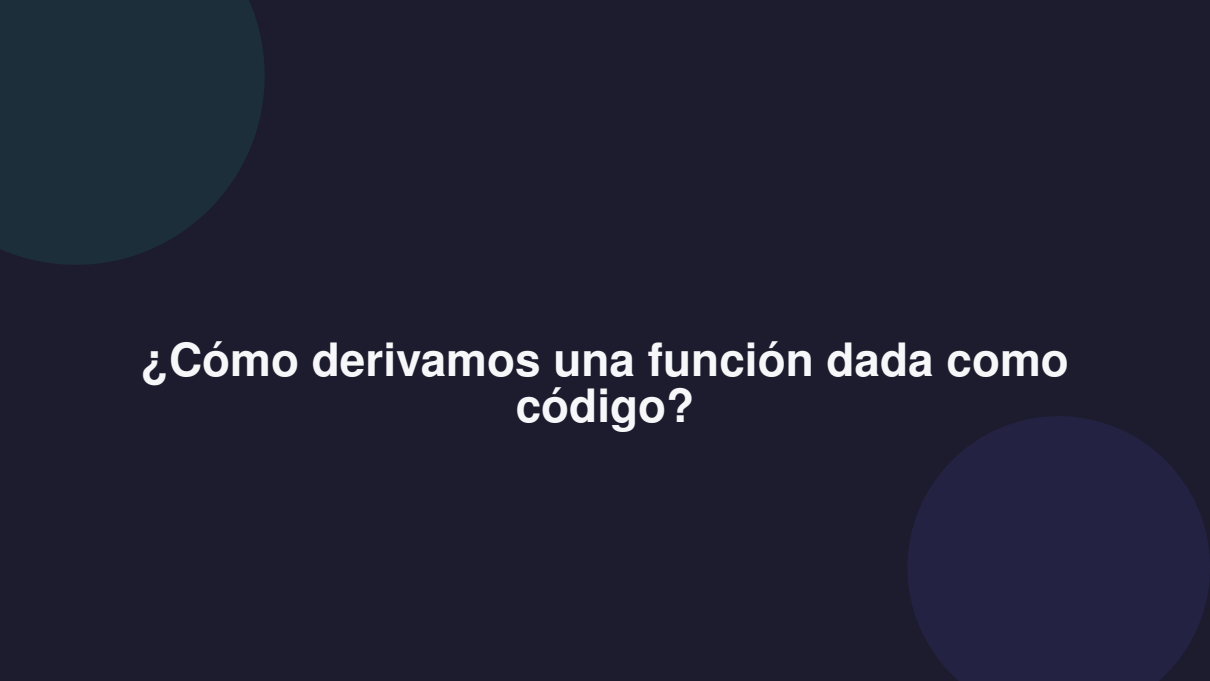


Programación Científica · UNAL · 2026-1

Diferenciación Automática

Módulo 3

El grafo computacional · JAX



¿Cómo derivamos una función dada como código?

El problema

Tenemos una función definida como código:

$$\mathcal{L}(x) = (x^2 + 3x) \sin(x)$$

Queremos $\frac{d\mathcal{L}}{dx}$ en algún punto x_0 específico.

Pregunta

¿Cómo lo calcula un computador, sin que nosotros derivemos a mano?

Hay tres caminos...

Opción 1: diferencias finitas

Aproximar por definición de la derivada:

$$\frac{d\mathcal{L}}{dx}(x_0) \approx \frac{\mathcal{L}(x_0 + h) - \mathcal{L}(x_0)}{h}$$

Ventajas:

- ▶ Trivial de implementar: solo evaluamos $\mathcal{L}$ dos veces.
- ▶ Funciona con **cualquier** código.

Problemas:

- ▶ Error de truncamiento $O(h)$: si h es grande, la aproximación es mala.
- ▶ Error de cancelación: si h es muy pequeño, $\mathcal{L}(x_0 + h) - \mathcal{L}(x_0)$ pierde dígitos significativos.

Opción 2: diferenciación simbólica

Derivar la fórmula a mano (o con SymPy):

$$\frac{d\mathcal{L}}{dx} = (2x + 3) \sin(x) + (x^2 + 3x) \cos(x)$$

Ventajas:

- ▶ Resultado exacto (sin error numérico).

Problemas:

- ▶ Hay que tener la **fórmula**: no sirve si $\mathcal{L}$ es un programa con condicionales, bucles, llamadas a funciones externas.
- ▶ **Expression swell**: derivar funciones compuestas produce expresiones que crecen explosivamente.
- ▶ Hay que re-derivar para cada nueva $\mathcal{L}$.

Ejemplo de swell: la derivada de $f(f(f(x)))$ con la regla de la cadena triplica el tamaño.

Opción 3: diferenciación automática (autodiff)

Idea

Aplicar la regla de la cadena **paso a paso sobre el código**, descomponiendo $\mathcal{L}$ en operaciones elementales cuyas derivadas conocemos.

Promesa:

- ▶ Exacto como la simbólica (sin error de truncamiento).
- ▶ Costo $O(1)$ evaluaciones de $\mathcal{L}$, sin importar el número de variables.
- ▶ Funciona con cualquier programa, no se necesita la fórmula.

Pregunta

¿Cómo puede ser simultáneamente exacto, barato y general?
La respuesta está en el **grafo computacional**.

El grafo computacional

Descomponer en operaciones elementales

Función:

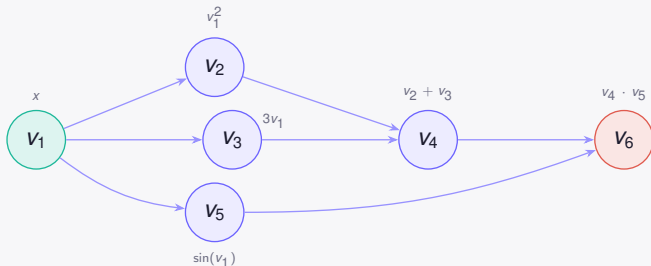
$$\mathcal{L}(x) = (x^2 + 3x) \sin(x)$$

Reescribirla introduciendo **variables intermedias** v_i , una por cada operación elemental:

Variable	Operación
$v_1 = x$	entrada
$v_2 = v_1^2$	potencia
$v_3 = 3 v_1$	escalar
$v_4 = v_2 + v_3$	suma
$v_5 = \sin(v_1)$	seno
$v_6 = v_4 \cdot v_5$	producto
$\mathcal{L} = v_6$	salida

El grafo

Cada v_i es un **nodo**. Cada dependencia $v_i \rightarrow v_j$ es una **arista**.



Propiedades del grafo

- ▶ **Dirigido**: cada arista tiene un sentido (de input hacia output).
- ▶ **Acíclico**: nunca regresa sobre sí mismo.
- ▶ Los **nodos hoja** (sin padres) son los inputs.
- ▶ Los **nodos raíz** (sin hijos) son las salidas.

Estrategia general

Derivar $\mathcal{L}$ respecto a x es **propagar derivadas** por las aristas, aplicando la regla de la cadena en cada nodo.

Hay dos maneras de hacer esa propagación: hacia adelante (forward) o hacia atrás (reverse).



Forward mode: propagar tangentes

Idea: arrastrar la derivada con cada valor

Junto a cada v_i propagamos su derivada respecto a la entrada:

$$\dot{v}_i = \frac{\partial v_i}{\partial x}$$

Inicialización: en la entrada, $\dot{v}_1 = \dot{x} = 1$.

Propagación: para cada nodo $v_j = g(v_i, \dots)$, aplicamos la regla de la cadena:

$$\dot{v}_j = \sum_{i \in \text{padres}(j)} \frac{\partial v_j}{\partial v_i} \dot{v}_i$$

Pasada forward

Recorremos el grafo **en el orden natural** (de entradas a salida), calculando v_i y $\dot{v}_i$ al mismo tiempo en cada nodo.

Forward mode sobre $\mathcal{L}(x) = (x^2 + 3x) \sin(x)$

Evaluamos en $x = 2$. Inicializamos $\dot{v}_1 = 1$.

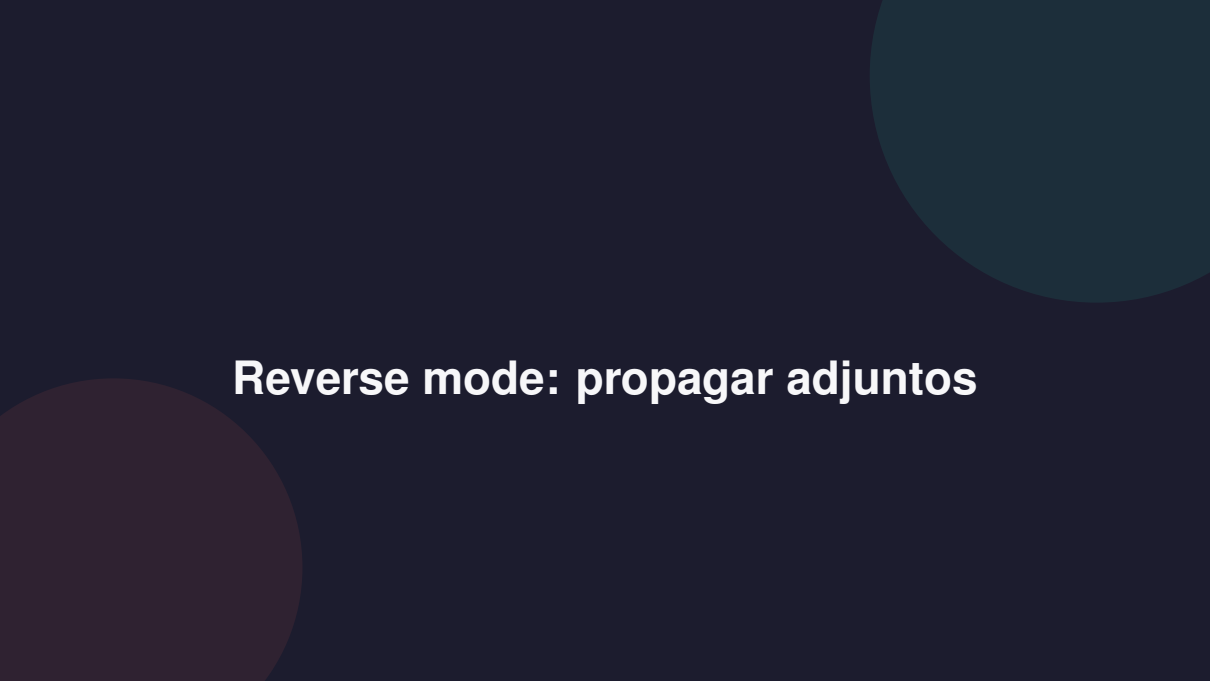
Nodo	Valor v_i	Tangente $\dot{v}_i$	Regla
$v_1 = x$	2	1	entrada
$v_2 = v_1^2$	4	$2v_1 \cdot \dot{v}_1 = 4$	cadena
$v_3 = 3v_1$	6	$3 \cdot \dot{v}_1 = 3$	cadena
$v_4 = v_2 + v_3$	10	$\dot{v}_2 + \dot{v}_3 = 7$	suma
$v_5 = \sin(v_1)$	$\sin 2 \approx 0.909$	$\cos(v_1) \cdot \dot{v}_1 = \cos 2 \approx -0.416$	cadena
$v_6 = v_4 \cdot v_5$	≈ 9.093	$\dot{v}_4 v_5 + v_4 \dot{v}_5 \approx 2.205$	producto
$\mathcal{L} = v_6$	≈ 9.093	$\dot{v}_6 \approx 2.205$	salida

Costo del forward mode

Una pasada forward produce $\frac{\partial \mathcal{L}}{\partial x}$ para **una** entrada x .

Si $\mathcal{L} : \mathbb{R}^n \rightarrow \mathbb{R}$ (muchas entradas, una salida):

- ▶ Necesitamos n pasadas forward (una por cada $\partial \mathcal{L} / \partial x_i$).
- ▶ Costo total: $O(n)$ veces el costo de evaluar $\mathcal{L}$.



Reverse mode: propagar adjuntos

Cambio de perspectiva

En lugar de propagar $\dot{v}_i = \frac{\partial v_i}{\partial x}$ hacia adelante, propagamos:

$$\bar{v}_i = \frac{\partial \mathcal{L}}{\partial v_i}$$

hacia atrás (de la salida hacia las entradas). El símbolo $\bar{v}$ se llama **adjunto** de v .

Regla de acumulación

Para cada nodo v_i con hijos v_j :

$$\bar{v}_i = \sum_{j \in \text{hijos}(i)} \bar{v}_j \cdot \frac{\partial v_j}{\partial v_i}$$

Pasada reverse: cómo se ejecuta

Paso 1 — Forward: evaluar v_i en todos los nodos y guardar.

Paso 2 — Inicializar el adjunto de salida:

$$\bar{\mathcal{L}} = 1$$

Paso 3 — Backward: recorrer el grafo **en orden inverso**, aplicando la regla de acumulación.

Resultado clave

Una sola pasada backward produce $\bar{v}_i$ para **todos** los nodos del grafo.

Costo: dos veces el costo de evaluar $\mathcal{L}$, independiente de n .

Reverse mode sobre $\mathcal{L}(x) = (x^2 + 3x) \sin(x)$

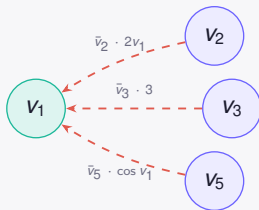
Forward (en $x = 2$) ya nos dio los v_i . Ahora, hacia atrás, con $\bar{v}_6 = 1$:

Adjunto	Cálculo	Valor
$\bar{v}_6 = \bar{\mathcal{L}}$	inicialización	1
$\bar{v}_5 = \bar{v}_6 \cdot \frac{\partial v_6}{\partial v_5} = \bar{v}_6 \cdot v_4$	producto	$1 \cdot 10 = 10$
$\bar{v}_4 = \bar{v}_6 \cdot \frac{\partial v_6}{\partial v_4} = \bar{v}_6 \cdot v_5$	producto	$1 \cdot 0.909 \approx 0.909$
$\bar{v}_3 = \bar{v}_4 \cdot 1$	suma	≈ 0.909
$\bar{v}_2 = \bar{v}_4 \cdot 1$	suma	≈ 0.909
$\bar{v}_1 = \bar{v}_2 \cdot 2v_1 + \bar{v}_3 \cdot 3 + \bar{v}_5 \cdot \cos(v_1)$	tres hijos	≈ 2.205

Resultado: $\left. \frac{d\mathcal{L}}{dx} \right|_{x=2} = \bar{v}_1 \approx 2.205.$ Coincide con forward y con el cálculo analítico.

El paso clave: v_1 tiene tres hijos

$$\bar{v}_1 = \underbrace{\bar{v}_2 \cdot 2v_1}_{\text{vía } v_2} + \underbrace{\bar{v}_3 \cdot 3}_{\text{vía } v_3} + \underbrace{\bar{v}_5 \cdot \cos(v_1)}_{\text{vía } v_5}$$



Esto es la regla de la cadena multivariada

Cada camino contribuye su propia derivada. Reverse mode **automatiza la suma**.

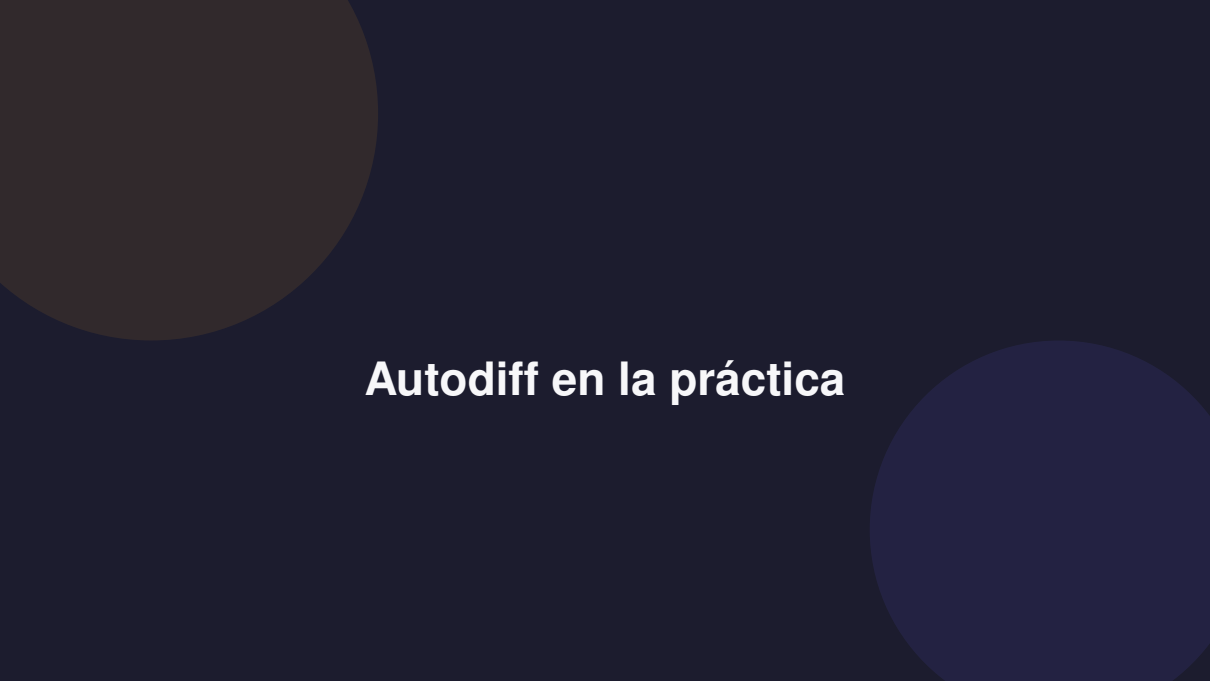
Forward vs. reverse: cuándo usar cada uno

Modo	Eficiente cuando	Costo
Forward	pocas entradas, muchas salidas	$O(n)$ pasadas
Reverse	muchas entradas, una salida	$O(1)$ pasadas

Reverse mode

Cuando $\mathcal{L}$ es una función escalar y x son los parámetros, reverse mode es el algoritmo que hace posible las derivadas.

En la práctica, autodiff casi siempre se refiere a reverse mode. JAX, PyTorch, TensorFlow lo usan por defecto con `grad`.



Autodiff en la práctica

jax.grad en una línea

```
import jax
import jax.numpy as jnp

def L(x):
    return (x**2 + 3*x) * jnp.sin(x)

dL_dx = jax.grad(L)

print(L(2.0))      # 9.0929...
print(dL_dx(2.0))  # 2.2052...
```

Lo que ocurre internamente

1. JAX construye el grafo computacional de L la primera vez que se llama.
2. Aplica reverse mode automáticamente para producir dL_{dx} .
3. Lo que escribimos a mano en la tabla, JAX lo hace por nosotros.

Ejercicio: programar autodiff en JAX

Función con for e if:

```
import jax
import jax.numpy as jnp

def f(x):
    y = 0.0
    for k in range(1, 4):
        if k % 2 == 0:
            y = y + (x**k) / k
        else:
            y = y - (x**k) / k
    return jnp.sin(y) * jnp.exp(-x**2 / 4)
```

Clave

El if depende del índice k , no del valor de x . El grafo computacional es el mismo para todo x , así que `jax.grad` funciona sin ningún cambio.

Tarea

1. Evaluar $f(1.5)$ y `jax.grad(f)(1.5)`.
2. Derivar f a mano y comparar.
3. Comparar con diferencias centradas para $h \in \{10^{-1}, 10^{-5}, 10^{-10}, 10^{-15}\}$.
4. ¿Qué pasaría si la condición fuera `if x > 0?`

Resultado esperado en $x = 1.5$:

```
f(1.5)           = -0.568356
jax.grad(f)(1.5) = 0.355733
```

Recapitulando

Grafo computacional

toda función es una composición de operaciones elementales

Reverse mode

una pasada hacia atrás → gradiente completo

`mbastidaso@unal.edu.co`